W5

Association, Aggregation, และ Composition

Data Structures & Object-Oriented Programming



Table of Contents

- ความสำคัญของการแยกคลาส
- การเก็บ Object เป็น Attribute
- Association
- Aggregation
- Composition
- Common Mistakes

บทนำ

- นิสิตได้เรียนรู้การสร้าง class เดี่ยว ที่สมบูรณ์ในตัวเอง
  - สัปดาห์ที่ 1 คือ Class และ Object พื้นฐาน
  - สัปดาห์ที่ 2 คือ Encapsulation ที่ป้องกันข้อมูลภายใน class เดียวนั้น
  - สัปดาห์ที่ 3 และ 4 คือ Inheritance และ Polymorphism ที่เชื่อมโยง class เข้าด้วยกันในรูปแบบ แม่-ลูก ผ่านความสัมพันธ์แบบ is-a เช่น SavingsAccount เป็น BankAccount ประเภทหนึ่ง
- ในโลกจริง ระบบซอฟต์แวร์ไม่ได้ประกอบด้วย hierarchy แม่-ลูกเพียงอย่างเดียว
  - ระบบร้านค้าออนไลน์ทั่วไป Order ไม่ได้ “เป็น” Customer และ Customer ก็ไม่ได้ “เป็น” Order
  - ทั้งสองเป็น class ที่เป็นอิสระจากกันโดยสิ้นเชิง แต่ ต้องทำงานร่วมกัน เพราะคำสั่งซื้อหนึ่งใบต้องรู้ว่าใครเป็นคนสั่ง ต้องส่งของไปที่ไหน และมีสินค้าอะไรอยู่ในนั้นบ้าง
- เมื่อ class สองตัวไม่มีความสัมพันธ์แบบ is-a ต่อกัน จะให้พวกมันทำงานร่วมกันได้อย่างไร
  - คำตอบ คือ ความสัมพันธ์ 3 แบบที่เรียกว่า Association, Aggregation, และ Composition

เมื่อพยายามยัดทุกอย่างไว้ใน Class เดียว

class Order สร้างขึ้นเพื่อจัดการคำสั่งซื้อในร้านค้าออนไลน์ และ ทำงานได้ถูกต้องสมบูรณ์ ไม่มี bug คำนวณราคารวมถูกต้อง แสดงข้อมูลได้ครบถ้วน

```python
class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.customer_name = ""
        self.customer_phone = ""
        self.shipping_street = ""
        self.shipping_city = ""
        self.shipping_zipcode = ""
        self.item_names = []       # list ของ string
        self.item_prices = []      # list ของ float
        self.item_quantities = [] # list ของ int

    def set_customer(self, name: str, phone: str) -> None:
        self.customer_name = name
        self.customer_phone = phone

    def set_shipping_address(self, street: str, city: str, zipcode: str):
        self.shipping_street = street
        self.shipping_city = city
        self.shipping_zipcode = zipcode

    def add_item(self, name: str, price: float, quantity: int) -> None:
        self.item_names.append(name)
        self.item_prices.append(price)
        self.item_quantities.append(quantity)
```

```python
    def calculate_total(self) -> float:
        total = 0.0
        for i in range(len(self.item_names)):
            total += self.item_prices[i] * self.item_quantities[i]
        return total

    def display_shipping_label(self) -> None:
        print(f"ส่งถึง: {self.customer_name}")
        print(f"ที่อยู่: {self.shipping_street}, {self.shipping_city}
{self.shipping_zipcode}")
```

ดูผิวเผิน class นี้ก็ใช้งานได้ตามปกติ

```python
order = Order("ORD-001")
order.set_customer("Nana", "081-234-5678")
order.set_shipping_address("123 ถ.สุขุมวิท", "กรุงเทพฯ", "10110")
order.add_item("เสื้อยืด", 250.0, 2)
order.add_item("กางเกงยีนส์", 890.0, 1)

print(order.calculate_total())        # 1390.0
order.display_shipping_label()
```

เมื่อพยายามยัดทุกอย่างไว้ใน Class เดียว

ปัญหาของ class นี้ คือ โครงสร้างข้อมูลภายในไม่ได้ใช้ประโยชน์จาก OOP เลยแม้แต่น้อย

ข้อมูลของลูกค้า ที่อยู่จัดส่ง และรายการสินค้า ถูกเก็บเป็น string, float และ list ธรรมดาที่ปะปนกันอยู่ใน Order

ทั้งที่แต่ละส่วนเหล่านี้ ควรมีพฤติกรรมและกฎเกณฑ์ของตัวเอง

```
class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.customer_name = ""
        self.customer_phone = ""
        self.shipping_street = ""
        self.shipping_city = ""
        self.shipping_zipcode = ""
        self.item_names = []      # list ของ string
        self.item_prices = []     # list ของ float
        self.item_quantities = [] # list ของ int
```

```
order = Order("ORD-001")
order.set_customer("Nana", "081-234-5678")
order.set_shipping_address("123 ถ.สุขุมวิท", "กรุงเทพฯ", "10110")
order.add_item("เสื้อยืด", 250.0, 2)
order.add_item("กางเกงยีนส์", 890.0, 1)

print(order.calculate_total())        # 1390.0
order.display_shipping_label()
```

Case 1 : ตรวจสอบความถูกต้องของที่อยู่

สมมติร้านค้าต้องการเพิ่มการตรวจสอบว่ารหัสไปรษณีย์ต้องมี 5 หลักเท่านั้น

สิ่งที่ต้องทำคือเปิด Order แล้วเพิ่ม logic การตรวจสอบใน set_shipping_address()

```python
def set_shipping_address(self, street: str, city: str, zipcode: str):
    self.shipping_street = street
    self.shipping_city = city
    self.shipping_zipcode = zipcode
```

```python
def set_shipping_address(self, street: str, city: str, zipcode: str) -
    if len(zipcode) != 5 or not zipcode.isdigit():
        raise ValueError("รหัสไปรษณีย์ต้องเป็นตัวเลข 5 หลัก")
    self.shipping_street = street
    self.shipping_city = city
    self.shipping_zipcode = zipcode
```

คำถาม คือ “ที่อยู่” ไม่ใช่สิ่งที่ใช้แค่กับ Order เพียงอย่างเดียว ร้านค้าอาจต้องการเก็บที่อยู่ของ Customer เองด้วย (ที่อยู่เรียกเก็บเงิน) หรือเก็บที่อยู่ของโกดังสินค้าหลายแห่ง แต่ logic การตรวจสอบรหัสไปรษณีย์ถูกฝังอยู่ใน Order โดยเฉพาะ จึงไม่มีทางนำ logic นี้ไปใช้ซ้ำกับที่อื่นได้เลย นอกจากจะ copy code ชุดเดิมไปวางซ้ำในทุกที่ที่ต้องเก็บที่อยู่

Case 2 : เพิ่มส่วนลดต่อรายการสินค้า

สมมติร้านค้าต้องการให้สินค้าบางรายการมีส่วนลดเฉพาะตัว (เช่น “เสื้อยืด” ลด 10%)

สิ่งที่ต้องทำคือเปิด Order แล้วต้องเพิ่ม list ใหม่อีกหนึ่งตัวคือ item_discounts และแก้ไข add_item() กับ calculate_total()

```
def add_item(self, name: str, price: float, quantity: int)
    self.item_names.append(name)
    self.item_prices.append(price)
    self.item_quantities.append(quantity)
```

```
def calculate_total(self) -> float:
    total = 0.0
    for i in range(len(self.item_names)):
        total += self.item_prices[i] * self.item_quantities[i]
    return total
```

```
def add_item(self, name, price, quantity: int, discount: float = 0.0):
    self.item_names.append(name)
    self.item_prices.append(price)
    self.item_quantities.append(quantity)
    self.item_discounts.append(discount)   # ต้องเพิ่ม list ใหม่ใน __init__ ด้วย
```

```
def calculate_total(self) -> float:
    total = 0.0
    for i in range(len(self.item_names)):
        item_total = self.item_prices[i] * self.item_quantities[i]
        item_total *= (1 - self.item_discounts[i])   # ต้องแก้สูตรตรงนี้ด้วย
        total += item_total
    return total
```

สังเกตว่าการเพิ่ม attribute หนึ่งตัวให้กับ “สินค้าหนึ่งรายการ” ต้องแก้ไข 4 จุดพร้อมกัน คือ

- **เพิ่ม list ใหม่ใน __init__, แก้ add_item(), แก้ calculate_total(), และถ้ามีที่อื่นที่ใช้ข้อมูลนี้ก็ต้องแก้ตามไปด้วย**

ความเสี่ยงที่ชัดเจนที่สุดคือ การใช้ index (i) เชื่อมโยงข้อมูลระหว่าง list 4 ตัว ถ้า list ใดตัวหนึ่งมีจำนวนสมาชิกไม่ตรงกัน (เช่น ลืมเพิ่มค่าใน item_discounts ตอนเรียก add_item() แบบเก่าที่ไม่ระบุ discount) โปรแกรมจะ error หรือแย่กว่านั้นคือให้ผลลัพธ์ผิดโดยไม่มี error ใดๆ

Case 3 : แสดงรายการสินค้าแบบละเอียด

สมมติร้านค้าต้องการฟังก์ชันแสดงใบเสร็จที่มีรายละเอียดสินค้าแต่ละชิ้นครบถ้วน

สิ่งที่ต้องทำคือเขียน method ใหม่ที่ต้องวน loop ผ่าน index เดียวกันอีกครั้ง

```
def display_receipt(self) -> None:
    for i in range(len(self.item_names)):
        name = self.item_names[i]
        price = self.item_prices[i]
        qty = self.item_quantities[i]
        discount = self.item_discounts[i]
        item_total = price * qty * (1 - discount)
        print(f"{name} x{qty} @ {price:.2f} (ลด {discount*100:.0f}%) =
{item_total:.2f}")
```

การเขียน code ลักษณะนี้ซ้ำแล้วซ้ำเล่าทุกครั้งที่ต้องการเข้าถึงข้อมูล “สินค้าหนึ่งรายการ” คือ สัญญาณที่ชัดเจนว่า “สินค้าหนึ่งรายการ” ควรเป็น object ของตัวเอง ไม่ใช่ข้อมูลที่กระจัดกระจายอยู่ใน list 4 ตัวที่ต้องเชื่อมโยงกันด้วย index

Solution : ให้ Object ประกอบกันเป็น Object

“ที่อยู่จัดส่ง” และ “สินค้าหนึ่งรายการ” ควรเป็น class ที่มี attribute และ method เฉพาะของมันเอง

แล้วให้ Order เก็บ object เหล่านี้ไว้เป็นส่วนประกอบ แทนที่จะเก็บข้อมูลดิบกระจัดกระจาย

```python
class Address:
    def __init__(self, street: str, city: str, zipcode: str):
        if len(zipcode) != 5 or not zipcode.isdigit():
            raise ValueError("รหัสไปรษณีย์ต้องเป็นตัวเลข 5 หลัก")
        self.street = street
        self.city = city
        self.zipcode = zipcode

    def display(self) -> str:
        return f"{self.street}, {self.city} {self.zipcode}"
```

```python
class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.shipping_address = None    # จะเก็บ Address object
        self.items = []                 # จะเก็บ OrderItem objects

    def set_shipping_address(self, address: Address) -> None:
        self.shipping_address = address

    def add_item(self, item: OrderItem) -> None:
        self.items.append(item)

    def calculate_total(self) -> float:
        return sum(item.calculate_subtotal() for item in self.items)
```

```python
class OrderItem:
    def __init__(self, name, price: float, quantity: int, discount = 0.0):
        self.name = name
        self.price = price
        self.quantity = quantity
        self.discount = discount

    def calculate_subtotal(self) -> float:
        return self.price * self.quantity * (1 - self.discount)

    def display(self) -> str:
        return (f"{self.name} x{self.quantity} @ {self.price:.2f} "
                f"(ลด {self.discount*100:.0f}%) =
{self.calculate_subtotal():.2f}")
```

เมื่อโครงสร้างเปลี่ยน ทั้ง 3 case จะเป็นเรื่องง่ายทันที

- การตรวจสอบรหัสไปรษณีย์อยู่ใน Address ที่เดียว นำไปใช้กับ Customer หรือที่ใดก็ได้ โดยไม่ต้อง copy code
- การเพิ่ม discount เป็นเพียงการเพิ่ม attribute หนึ่งตัวใน OrderItem ไม่ต้องแก้ไขหลายจุดพร้อมกัน
- การแสดงรายละเอียดสินค้าก็เป็นเพียงการเรียก item.display() ผ่าน loop เดียว

หัวใจของสัปดาห์นี้

- สังเกตว่า Order เก็บทั้ง Address และ OrderItem เป็น attribute เหมือนกัน แต่ทั้งคู่มีธรรมชาติที่ต่างกันโดยสิ้นเชิง
  - Address ถูกสร้างขึ้นจากภายนอกก่อน (Address("123 ถ.สุขุมวิท", "กรุงเทพฯ", "10110")) แล้วจึงส่งเข้าไปให้ Order ผ่าน set_shipping_address()
    - คำถามคือถ้า Order ถูกยกเลิกหรือลบทิ้ง Address object นั้นควรจะยังมีความหมายอยู่หรือไม่ (เช่น เอาไปใช้กับคำสั่งซื้อใบอื่นได้หรือไม่)
  - OrderItem ให้ลองคิดดูว่า “รายการสินค้าหนึ่งบรรทัดในใบสั่งซื้อ” มีความหมายอะไร
    - มันมีตัวตนเป็นของตัวเองในระบบจริงหรือไม่ หรือมันมีความหมายก็ต่อเมื่อเป็นส่วนหนึ่งของ order เท่านั้น
- Aggregation และ Composition คือ รูปแบบความสัมพันธ์ที่บอกว่า object หนึ่ง “มี” อีก object หนึ่งอยู่ภายใน แต่มีระดับความผูกพันที่ต่างกัน
  - การเลือกใช้แบบที่ถูกต้องจะส่งผลโดยตรงต่อว่า object เหล่านั้นควรถูกสร้าง ถูกส่งต่อ และถูกทำลายอย่างไรในระบบจริง

การเก็บ Object เป็น Attribute

การเก็บ Object ไว้เป็น Attribute

- ที่ผ่านมา attribute ทุกตัวที่นิสิตเคยเขียน เก็บค่าประเภทพื้นฐาน (primitive types) เสมอ
- แต่ในความเป็นจริง attribute ของ class เก็บอะไรก็ได้ที่เป็น object ใน Python
- เนื่องจากทุก instance ที่สร้างจาก class ก็คือ object เราจึงสามารถ เขียนแบบนี้

```python
class Engine:
    def __init__(self, horsepower: int):
        self.horsepower = horsepower

    def start(self) -> None:
        print(f"เครื่องยนต์ {self.horsepower} แรงม้า สตาร์ทติดแล้ว")
```

```python
class Car:
    def __init__(self, brand: str, horsepower: int):
        self.brand = brand
        self.engine = Engine(horsepower)   # attribute นี้เก็บ "object" ไม่ใช่ค่าพื้นฐาน

    def start_engine(self) -> None:
        print(f"{self.brand}: ")
        self.engine.start()    # เรียก method ของ object ที่เก็บไว้ใน attribute
```

```python
car = Car("Toyota", 150)
car.start_engine()
```

```
Toyota:
เครื่องยนต์ 150 แรงม้า สตาร์ทติดแล้ว
```

- self.engine เก็บ object ทั้งก้อน ที่มี attribute และ method ของตัวเอง
- เมื่อต้องการใช้งาน ก็เข้าถึงผ่าน self.engine.start()

Object Reference

เมื่อตัวแปรสองตัว “ชี้ไปที่ object เดียวกัน” การแก้ไขผ่านตัวแปรหนึ่งจะส่งผลกระทบเมื่อเข้าถึงผ่านอีกตัวแปรหนึ่งด้วยเสมอ

```
engine1 = Engine(150)
engine2 = engine1
# engine2 ไม่ได้ "คัดลอก" object ใหม่ แต่ชี้ไปที่ object เดียวกับ engine1

engine2.horsepower = 999
print(engine1.horsepower)    # 999 — engine1 ก็เปลี่ยนตามด้วย!
```

- ปรากฏการณ์นี้เกิดขึ้นเพราะตัวแปรใน Python ไม่ได้ “เก็บ object” โดยตรง
  - แต่เก็บที่อยู่ในหน่วยความจำที่ object นั้นอาศัยอยู่
  - เมื่อเขียน engine2 = engine1 จึงเป็นการคัดลอก “ที่อยู่” ไม่ใช่คัดลอกตัว object เอง
- ทั้ง engine1 และ engine2 จึงชี้ไปยัง object เดียวกันในหน่วยความจำ
  - การแก้ไขผ่านตัวแปรใดตัวแปรหนึ่งจึงกระทบอีกตัวแปรเสมอ

Object Reference (2)

แตกต่างค่าพื้นฐานที่คุ้นเคย ลองเทียบกับ int

```
x = 5
y = x
y = 10
print(x)    # 5 — x ไม่เปลี่ยนตาม
```

- int, str, float, bool ใน Python มีพฤติกรรมในลักษณะ “คัดลอกค่า” ทุกครั้งที่กำหนดใหม่
  - เพราะเป็น immutable object – แก้ไขค่าเดิมไม่ได้ ทำได้แค่สร้างค่าใหม่มาแทน
- แต่ object ที่สร้างจาก class มีพฤติกรรมตรงข้าม คือ mutable และสามารถถูกแก้ไขผ่านตัวแปรใดก็ได้ที่ชี้ไปหามัน
- กลไกนี้สำคัญมากเพราะเป็นรากฐานของทั้ง Aggregation และ Composition
  - “ถ้า object ตัวหนึ่งถูกส่งเข้าไปใน object อื่น แล้ว object ต้นทางถูกแก้ไขทีหลัง จะกระทบ object ที่รับไปหรือไม่”
    - คำตอบ คือ เปลี่ยน เพราะว่าเป็น object reference มันชี้ไปที่ object เดียวกัน

Attribute ที่ยังไม่มีค่า — รูปแบบ None

- บ่อยครั้งที่ class หนึ่งควรมี attribute ที่เก็บ object ของอีก class
- แต่ ณ ตอนสร้าง object ยังไม่รู้ว่าจะเป็น object ตัวไหน
  - เช่น Order ที่ยังไม่รู้ที่อยู่จัดส่ง ตอนสร้างขึ้นมาแรกๆ
- รูปแบบที่ใช้กันทั่วไป คือ กำหนดค่าเริ่มต้นเป็น None ไว้ก่อน แล้วค่อยกำหนดค่าจริงในภายหลังผ่าน method

```
class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.shipping_address = None    # ยังไม่มี Address ตอนสร้าง

    def set_shipping_address(self, address: Address) -> None:
        self.shipping_address = address
```

Attribute ที่ยังไม่มีค่า — รูปแบบ None

- ข้อควรระวังที่สำคัญมากคือ ถ้าพยายามเข้าถึง attribute ที่ยังเป็น None ผ่าน dot chaining จะเกิด error ทันที

```
order = Order("ORD-001")
print(order.shipping_address.city)
# AttributeError: 'NoneType' object has no attribute 'city'
```

Error นี้เกิดขึ้นเพราะ order.shipping_address มีค่าเป็น None ซึ่งไม่มี attribute ชื่อ city อยู่เลย
การเขียน code ที่ปลอดภัยจึงต้องตรวจสอบก่อนเสมอว่า attribute นั้นมีค่าจริงหรือยัง ก่อนจะเข้าถึงด้วย dot

```
def display_shipping_info(self) -> None:
    if self.shipping_address is None:
        print("ยังไม่ได้ระบุที่อยู่จัดส่ง")
    else:
        print(f"จัดส่งไปที่: {self.shipping_address.display()}")
```

รูปแบบการตรวจสอบ is None ก่อนเข้าถึง attribute ที่เก็บ object ควรทำเสมอเมื่อเขียน code ที่มีความสัมพันธ์ระหว่าง class

List ของ Object

attribute สามารถเก็บเป็น list ของ object ได้

```python
class Order:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.items: list[OrderItem] = []   # list ที่เก็บ OrderItem หลายตัว

    def add_item(self, item: OrderItem) -> None:
        self.items.append(item)

    def calculate_total(self) -> float:
        # วน loop ผ่าน object แต่ละตัว แล้วเรียก method ของมันโดยตรง
        return sum(item.calculate_subtotal() for item in self.items)
```

- สังเกตว่า calculate_total() ไม่ต้องรู้เลยว่า OrderItem แต่ละตัวมี attribute อะไรบ้างภายใน มันแค่เรียก item.calculate_subtotal()
  - นี่คือประโยชน์โดยตรงของการเก็บข้อมูลเป็น object แทนที่จะเป็น list ของค่าพื้นฐาน
  - ทุกอย่างที่เกี่ยวกับ “สินค้าหนึ่งรายการ” อยู่รวมกันใน object เดียว

บาง object ควรถูกสร้างขึ้นภายใน “เจ้าของ” มันเท่านั้น

```
item1 = OrderItem("เสื้อยืด", 250.0, 2)

order_a = Order("ORD-A")
order_a.add_item(item1)

order_b = Order("ORD-B")
order_b.add_item(item1)    # เพิ่ม item1 ตัวเดิมเข้าไปใน order_b ด้วย

item1.price = 500.0         # แก้ไขราคาผ่านตัวแปร item1 โดยตรง

print(order_a.items[0].price)    # ?
print(order_b.items[0].price)    # ?
```

- ทั้งสองบรรทัดพิมพ์ 500.0 เหมือนกัน เพราะ order_a.items[0] และ order_b.items[0] และ item1 ชี้ไปยัง object เดียวกัน ในหน่วยความจำ
  - การแก้ไขผ่านชื่อใดก็ตามจึงเห็นผลเหมือนกันหมดเมื่อเข้าถึงผ่านชื่ออื่น
- ในทางธุรกิจจริง “รายการสินค้าหนึ่งบรรทัดในคำสั่งซื้อ” ควรถูกใช้ร่วมกันระหว่างสอง order พร้อมกันแบบนี้หรือไม่ หรือควรจะเป็นคนละ object กันแม้จะมีข้อมูลเหมือนกัน (ชื่อสินค้า ราคา จำนวน)
- Composition => กำหนดว่า object ถูกสร้างขึ้นภายใน “เจ้าของ” ของมันเท่านั้น

Association

Association

```python
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self.owner = owner
        self.__balance = balance

    def withdraw(self, amount: float) -> None:
        if amount > self.__balance:
            raise ValueError("ยอดเงินไม่เพียงพอ")
        self.__balance -= amount

    def deposit(self, amount: float) -> None:
        self.__balance += amount

    def transfer(self, amount: float, target_account: "BankAccount") -> None:
        self.withdraw(amount)
        target_account.deposit(amount)    # ← เรียก method ของ object อื่นที่รับเข้ามา
```

```python
alice = BankAccount("Alice", 1000)
bob = BankAccount("Bob", 500)

alice.transfer(300, bob)
# ส่ง bob (object) เข้าไปเป็น parameter
```

def transfer(self, amount: float, target_account: "BankAccount")

- target_account เป็น object ของ class BankAccount ที่ถูกส่งเข้ามาเป็น parameter แล้ว transfer() ก็เรียก target_account.deposit(amount) เพื่อสั่งให้ object นั้นทำงานบางอย่าง
- นี่คือตัวอย่างของ Association — ความสัมพันธ์ระหว่าง object ที่ “ใช้งานกันชั่วคราว” โดยไม่มีความเป็นเจ้าของต่อกันเลย

นิยามของ Association

- Association คือ ความสัมพันธ์ที่ class หนึ่ง “รู้จักและใช้งาน” อีก class หนึ่ง โดยไม่มีความเป็นเจ้าของ (ownership) ต่อกัน
  - ทั้งสอง object เป็นอิสระต่อกันอย่างสมบูรณ์ ทั้งก่อนและหลังการใช้งานร่วมกัน
- จุดสังเกตสำคัญของ Association คือ object ที่ถูกใช้งานไม่ได้ถูก “เก็บ” ไว้เป็น attribute อย่างถาวร
  - มันถูกใช้ชั่วคราวภายใน method แล้วก็จบไป

```python
def transfer(self, amount: float, target_account: "BankAccount") -> None:
    self.withdraw(amount)
    target_account.deposit(amount)
```

เมื่อจบ method target_account ไม่ได้ถูกเก็บไว้ที่ไหนใน self เลย

Association ในรูปแบบอื่นที่พบบ่อย

- รูปแบบ 1 : Method คืนค่าเป็น Object เช่น method ที่ค้นหาและคืน object กลับมาให้ผู้เรียกใช้ต่อ

```python
class Library:
    def __init__(self):
        self.__books: list[Book] = []

    def find_book_by_title(self, title: str) -> "Book | None":
        for book in self.__books:
            if book.title == title:
                return book    # คืน object กลับไป — เป็น Association
        return None
```

- รูปแบบ 2: Local Variable ภายใน Method เช่น การสร้าง object ชั่วคราวขึ้นมาใช้งานแล้วทิ้งไปโดยไม่เก็บไว้เป็น attribute

```python
class OrderProcessor:
    def validate_order(self, order: "Order") -> bool:
        validator = OrderValidator()    # สร้างขึ้นมาใช้ชั่วคราว
        return validator.check(order)   # ใช้เสร็จแล้วก็จบ ไม่เก็บไว้
```

Multiplicity

- Association มักถูกกำกับด้วยตัวเลขสองด้าน เรียกว่า multiplicity เพื่อบอกว่า object แต่ละฝั่งเกี่ยวข้องกันกี่ตัว เช่น
  - “ลูกค้าหนึ่งคน สามารถทำ transfer ได้กับบัญชีปลายทางกี่บัญชี”
  - “บัญชีหนึ่งบัญชี ถูกใช้เป็นปลายทางการโอนได้จากกี่บัญชีต้นทาง”
- ในทาง code แนวคิดนี้สะท้อนออกมาเป็นคำถามว่า method หนึ่งควรรับ object เดียว หรือรับ list ของ object เข้ามา เช่น

```python
# รับ object เดียว — ความสัมพันธ์แบบ "หนึ่งต่อหนึ่ง" ในการเรียกครั้งนี้
def transfer(self, amount: float, target_account: "BankAccount") -> None:
    ...

# รับหลาย object พร้อมกัน — ความสัมพันธ์แบบ "หนึ่งต่อหลาย" ในการเรียกครั้งนี้
def send_to_multiple(self, amount: float, targets: list) -> None:
    for target in targets:
        self.transfer(amount, target)
```

Aggregation

Aggregation

- Aggregation คือ ความสัมพันธ์ที่ class หนึ่ง “มี” อีก class หนึ่งเป็นส่วนประกอบ โดยส่วนประกอบนั้นถูก เก็บไว้เป็น attribute อย่างถาวร
- สิ่งที่ทำให้ Aggregation ยังคงเป็นความสัมพันธ์แบบ “หลวม” คือ ส่วนประกอบนั้นดำรงอยู่ได้อย่างอิสระ แม้ไม่มี “เจ้าของ” แล้วก็ตาม
    - ส่วนประกอบมีตัวตนและความหมายเป็นของตัวเอง ไม่ได้พึ่งพาเจ้าของเพื่อให้มีความหมาย
- ตัวอย่าง เช่น ความสัมพันธ์ระหว่าง Team (ทีมฟุตบอล) กับ Player (นักฟุตบอล)
    - นักฟุตบอลคนหนึ่งเป็นสมาชิกของทีมได้ แต่ถ้าทีมนั้นถูกยุบไป นักฟุตบอลคนนั้นก็ยังมีตัวตนอยู่ในระบบ อาจย้ายไปทีมอื่น หรือกลายเป็น free agent
    - Player ไม่ได้หมดความหมายไปพร้อมกับ Team

สร้างระบบ Team และ Player

1) ออกแบบ Player เป็น class ที่สมบูรณ์ในตัวเอง ไม่ขึ้นกับว่าจะถูกนำไปใช้กับ Team หรือไม่

```python
class Player:
    def __init__(self, name: str, position: str):
        self.name = name
        self.position = position
        self.__goals = 0

    def score_goal(self) -> None:
        self.__goals += 1

    @property
    def goals(self) -> int:
        return self.__goals

    def display(self) -> str:
        return f"{self.name} ({self.position}) - ประตู: {self.__goals}"
```

สังเกตว่า Player ไม่มีการอ้างอิงถึง Team เลย มันสมบูรณ์ในตัวเอง และสามารถสร้าง object ทดสอบได้อิสระ

```python
messi = Player("Messi", "Forward")
messi.score_goal()
print(messi.display())    # Messi (Forward) - ประตู: 1
```

สร้างระบบ Team และ Player

2) ออกแบบ Team ให้เก็บ Player เป็น Aggregation

```python
class Team:
    def __init__(self, name: str):
        self.name = name
        self.__players: list[Player] = []  # private

    def add_player(self, player: Player) -> None:
        # player ถูกสร้างจากภายนอกเสร็จแล้ว จึงส่งเข้ามา — นี่คือ Aggregation
        self.__players.append(player)

    def remove_player(self, player: Player) -> None:
        if player in self.__players:
            self.__players.remove(player)
            print(f"{player.name} ออกจากทีม {self.name} แล้ว")
```

```python
@property
def player_count(self) -> int:
    return len(self.__players)

def display_roster(self) -> None:
    print(f"=== ทีม {self.name} ===")
    for player in self.__players:
        print(f"  {player.display()}")
```

- _players เป็น private attribute ตามหลัก Encapsulation ไม่เปิดให้ภายนอกเข้าถึง list ตรงๆ ได้
  - ต้องผ่าน add_player(), remove_player(), และ player_count เท่านั้น
- add_player() รับ Player object ที่สร้างเสร็จสมบูรณ์แล้วเข้ามาเป็น parameter ไม่ได้สร้าง Player ขึ้นมาเองภายใน method
  - นี่คือกลไกสำคัญที่สุดที่ทำให้ความสัมพันธ์นี้เป็น Aggregation

ทดสอบพฤติกรรมของ Aggregation

```
# Player ถูกสร้างขึ้นก่อน โดยไม่ผูกกับทีมใดเลย
messi = Player("Messi", "Forward")
suarez = Player("Suarez", "Forward")

# ค่อยนำไปผูกกับทีมทีหลัง
inter_miami = Team("Inter Miami")
inter_miami.add_player(messi)
inter_miami.add_player(suarez)

inter_miami.display_roster()
# === ทีม Inter Miami ===
#    Messi (Forward) - ประตู: 0
#    Suarez (Forward) - ประตู: 0

print(inter_miami.player_count)    # 2
```

พิสูจน์คุณสมบัติที่สำคัญที่สุดของ Aggregation คือ Player ยังมีชีวิตอิสระอยู่ได้แม้ Team จะหายไป

```
del inter_miami            # ลบ object ทีมทิ้ง

print(messi.name)          # Messi — ยังทำงานได้ปกติ!
messi.score_goal()
print(messi.goals)         # 1 — ยังใช้งานได้เหมือนไม่มีอะไรเกิดขึ้น
```

- แม้ inter_miami จะถูกลบไปแล้ว messi (object) ยังคงอยู่ในหน่วยความจำและใช้งานได้ตามปกติท
- เพราะตัวแปร messi ยังคงชี้ไปยัง object เดิมอยู่ ไม่ได้ถูกทำลายไปพร้อมกับ Team
  - นี่คือหลักฐานที่จับต้องได้ว่า Player เป็นอิสระจาก Team อย่างแท้จริง

พิสูจน์ว่า Player ย้ายทีมได้

คุณสมบัติสำคัญอีกอย่างของ Aggregation คือ object เดียวกันสามารถถูกใช้ร่วมกันหรือย้ายไปมาระหว่างเจ้าของหลายรายได้

```
messi = Player("Messi", "Forward")

barcelona = Team("Barcelona")
barcelona.add_player(messi)
print(barcelona.player_count)    # 1

# Messi ย้ายทีม
barcelona.remove_player(messi)
# → Messi ออกจากทีม Barcelona แล้ว

inter_miami = Team("Inter Miami")
inter_miami.add_player(messi)    # Player ตัวเดิม ถูกนำไปผูกกับทีมใหม่

print(barcelona.player_count)       # 0
print(inter_miami.player_count)     # 1
print(messi.goals)                  # ค่า goals เดิมยังคงอยู่ครบ ไม่ได้รีเซ็ตตอนย้ายทีม
```

- สังเกตว่า messi เป็น object ตัวเดียวกัน ตลอดทั้งกระบวนการ
- ไม่มีการสร้าง Player ใหม่ขึ้นมาเลยตอนย้ายทีม มีเพียงการเพิ่มและลบออกจาก list ภายใน Team เท่านั้น
- ข้อมูล goals ที่สะสมไว้จึงยังคงอยู่ครบถ้วนไม่ว่าจะย้ายไปกี่ทีมก็ตาม

เปรียบเทียบกับสิ่งที่ไม่ใช่ Aggregation

ลองดู code ที่ ดูคล้าย Aggregation แต่จริงๆ ไม่ใช่ เพราะสร้าง object ขึ้นภายในแทนที่จะรับจากภายนอก

```
class Team:
    def add_player(self, name: str, position: str) -> None:
        # ผิดเจตนา — สร้าง Player ใหม่ขึ้นภายในนี้เอง
        player = Player(name, position)
        self.__players.append(player)
```

- โค้ดนี้ ไม่ใช่ Aggregation เพราะ Player ไม่เคยมีตัวตนอยู่นอก Team เลย
- ทุกครั้งที่เรียก add_player() จะได้ Player ตัวใหม่ที่ผูกกับทีมนั้นทันทีตั้งแต่เกิด
  - ไม่มีทางให้ Player ตัวเดียวกันย้ายไปทีมอื่นได้ด้วย (เพราะไม่มีตัวแปรอื่นชี้ไปหา object นั้นนอกจาก Team เอง)

Encapsulation ยังคงสำคัญเสมอ

ข้อผิดพลาดที่พบบ่อยมากเมื่อเขียน Aggregation คือ การลืมหลัก Encapsulation โดยเฉพาะการเปิดให้เข้าถึง list ของ object ได้โดยตรงจากภายนอก

```
class Team:
    def __init__(self, name: str):
        self.name = name
        self.players: list[Player] = []    # ❌ public — อันตราย!
```

```
team = Team("Barcelona")
team.players.append(fake_player)
team.players.clear()
```

เพิ่ม player ได้โดยไม่ผ่าน add_player()
ลบผู้เล่นทั้งหมดได้โดยไม่มีการควบคุมใดๆ

- ถ้า players เป็น public attribute โค้ดภายนอกจะสามารถแก้ไข list นี้ได้โดยตรงโดยไม่ผ่าน method ที่ออกแบบไว้
  - ทำลายการควบคุมทั้งหมดที่ตั้งใจสร้างไว้
- วิธีแก้คือทำให้ _players เป็น private เสมอ

Encapsulation ยังคงสำคัญเสมอ

ข้อผิดพลาดที่พบบ่อยมากเมื่อเขียน Aggregation คือ การลืมหลัก Encapsulation โดยเฉพาะการเปิดให้เข้าถึง list ของ object ได้โดยตรงจากภายนอก

```
class Team:
    def __init__(self, name: str):
        self.name = name
        self.players: list[Player] = []    # ❌ public — อันตราย!
```

```
team = Team("Barcelona")
team.players.append(fake_player)
team.players.clear()
```

เพิ่ม player ได้โดยไม่ผ่าน add_player()
ลบผู้เล่นทั้งหมดได้โดยไม่มีการควบคุมใดๆ

- ถ้าต้องการเปิดให้ภายนอก “อ่าน” รายชื่อผู้เล่นได้ ควรทำผ่าน property ที่คืน copy ของ list แทนที่จะคืน reference ตรงๆ

```
@property
def players(self) -> list[Player]:
    return list(self.__players)    # คืน copy ไม่ใช่ list ตัวจริง
    # ป้องกันไม่ให้ภายนอกเอา list ที่ได้ไป .append() หรือ .clear() แล้วกระทบต้นฉบับ
```

Composition

Composition

- ความสัมพันธ์ที่ class หนึ่ง “ประกอบด้วย” อีก class หนึ่งอย่างแนบแน่นที่สุด
  - โดยส่วนประกอบนั้น ดำรงอยู่ไม่ได้เลยหากไม่มีเจ้าของ
- กลไกที่ทำให้เกิดความแนบแน่นระดับนี้ คือ ส่วนประกอบถูกสร้างขึ้นภายในเจ้าของโดยตรง ไม่ใช่ถูกสร้างจากภายนอกแล้วส่งเข้ามา
  - ผลที่ตามมาคือ ไม่มีทางที่ใครจะ “ยืม” ส่วนประกอบนี้ไปใช้ที่อื่น หรือสร้างมันขึ้นมาลอยๆ ก่อนแล้วค่อยผูกกับเจ้าของทีหลังได้
- ตัวอย่าง เช่น ความสัมพันธ์ระหว่าง House (บ้าน) กับ Room (ห้อง) ห้องหนึ่งห้องในบ้าน เช่น
  - “ห้องนอนใหญ่ ขนาด 20 ตร.ม.” ไม่มีความหมายอะไรเลยถ้าแยกออกจากบ้านหลังนั้น
  - ไม่มีใครพูดถึง “ห้องนอน” ลอยๆ โดยไม่อ้างอิงว่าเป็นห้องของบ้านหลังไหน
  - ถ้าบ้านถูกรื้อถอนไป ห้องนั้นก็หมดความหมายไปพร้อมกัน

การแยก Composition จาก Aggregation

- คำถาม >> “object ที่ถูกเก็บไว้ ถูกสร้างจากภายนอกแล้วส่งเข้ามา หรือถูกสร้างขึ้นภายใน”
- ใน Aggregation คำตอบ คือ จากภายนอก
  - add_player(player) รับ Player ที่สร้างเสร็จแล้วเข้ามา
- ใน Composition คำตอบ คือ ภายใน
  - method จะรับเพียง ข้อมูลดิบ (string, number) แล้ว สร้าง object ขึ้นเองภายใน method นั้น
  - ผู้เรียกใช้จากภายนอกไม่มีโอกาสสร้าง object ของส่วนประกอบนี้ได้เลยด้วยตัวเอง

สร้างระบบ House และ Room

1) ออกแบบ Room

```python
class Room:
    def __init__(self, name: str, area: float):
        self.name = name
        self.area = area

    def display(self) -> str:
        return f"{self.name} ({self.area} ตร.ม.)"
```

Room เป็น class ที่สมบูรณ์ในตัวเอง มี attribute และ method ของตัวเอง

ความแตกต่างไม่ได้อยู่ใน Room แต่อยู่ที่ ว่า House จะเลือกสร้าง Room อย่างไร

สร้างระบบ House และ Room

2) ออกแบบ House ให้สร้าง Room ขึ้นภายใน

```python
class House:
    def __init__(self, address: str):
        self.address = address
        self.__rooms: list[Room] = []   # private

    def add_room(self, name: str, area: float) -> None:
        # รับแค่ข้อมูลดิบ (name, area) ไม่ใช่ Room object สำเร็จรูป
        # Room ถูกสร้างขึ้น "ภายใน" method นี้เอง — นี่คือ Composition
        room = Room(name, area)
        self.__rooms.append(room)

    @property
    def room_count(self) -> int:
        return len(self.__rooms)

    @property
    def total_area(self) -> float:
        return sum(room.area for room in self.__rooms)

    def display_rooms(self) -> None:
        print(f"=== บ้านเลขที่ {self.address} ===")
        for room in self.__rooms:
            print(f"  {room.display()}")
```

เปรียบเทียบ signature ของ add_room() กับ add_player()

```python
# Aggregation (Team) — รับ object สำเร็จรูป
def add_player(self, player: Player) -> None:
    self.__players.append(player)

# Composition (House) — รับข้อมูลดิบ แล้วสร้าง object เอง
def add_room(self, name: str, area: float) -> None:
    room = Room(name, area)
    self.__rooms.append(room)
```

- ความต่างของ parameter type คือสัญญาณที่ชัดเจนที่สุดในการอ่าน code เพื่อแยกสองความสัมพันธ์นี้ออกจากกัน
- add_player() รับ Player (object)
- add_room() รับ str และ float (ข้อมูลดิบ) แล้วค่อยประกอบเป็น Room เองภายใน

สร้างระบบ House และ Room

ทดสอบพฤติกรรมของ Composition

```
house = House("123 ถ.สุขุมวิท")
house.add_room("ห้องนอนใหญ่", 20.0)
house.add_room("ห้องครัว", 12.0)
house.add_room("ห้องน้ำ", 6.0)

house.display_rooms()
# === บ้านเลขที่ 123 ถ.สุขุมวิท ===
#   ห้องนอนใหญ่ (20.0 ตร.ม.)
#   ห้องครัว (12.0 ตร.ม.)
#   ห้องน้ำ (6.0 ตร.ม.)

print(house.room_count)     # 3
print(house.total_area)     # 38.0
```

- ไม่มีบรรทัดไหนเลยที่เขียน room = Room(...) จากภายนอก House
  - ไม่มีทางสร้าง Room ลอยๆ ในระบบนี้ได้เลย
  - ทุกครั้งที่มีห้องเกิดขึ้นในระบบ มันต้องเกิดผ่าน house.add_room() เท่านั้น

สร้างระบบ House และ Room

พิสูจน์ว่า Room ตายไปพร้อม House

คุณสมบัติที่สำคัญที่สุดของ Composition — เมื่อ House หายไป Room ก็ไม่มีทางเข้าถึงได้อีกเลย

```
house = House("123 ถ.สุขุมวิท")
house.add_room("ห้องนอนใหญ่", 20.0)

# ไม่มีตัวแปรใดเลยที่ชี้ไปยัง Room object นั้นโดยตรง
# มีเพียง house ที่เก็บมันไว้ผ่าน __rooms (private)

del house    # ลบ House ทิ้ง
```

- ในตัวอย่าง Team/Player มีตัวแปร messi ที่ชี้ไปยัง Player object โดยตรงจากภายนอกเสมอ จึงยังเข้าถึงได้แม้ Team หายไป
- ใน House/Room ไม่มีใครสร้าง Room object แล้วเก็บตัวแปรชี้ไปหามันจากภายนอกเลย เพราะมันถูกสร้างและซ่อนอยู่ภายใน add_room() ทั้งหมด
- เมื่อ House ถูกทำลาย ไม่มีการอ้างอิงใดเหลืออยู่ให้เข้าถึง Room นั้นได้อีก และ Python’s garbage collector จะเก็บกวาดหน่วยความจำของ Room เหล่านั้นไป

ทดลองสร้าง Room ลอยๆ ดูว่าเกิดอะไรขึ้น

ถ้าพยายามสร้าง Room แล้วยัดเข้าไปใน House โดยไม่ผ่าน add_room()

```
lonely_room = Room("ห้องสมุด", 15.0)

house = House("456 ถ.พระราม 4")
```

สร้างได้ เพราะ Room class ไม่ได้ห้ามสร้างเอง

house._rooms.append(lonely_room) # ❌ ทำไม่ได้ – _rooms เป็น private

- Room class ไม่ได้มีกลไกป้องกันไม่ให้สร้าง object อิสระ
- Composition ไม่ใช่กฎที่ Python บังคับในระดับภาษา แต่เป็นวินัยในการออกแบบ interface ที่ผู้เขียน code ต้องรักษาไว้เอง
  - โดยการไม่เปิด constructor หรือ method ใดที่รับ object สำเร็จรูปของส่วนประกอบเข้ามาจากภายนอก

เปรียบเทียบ Aggregation กับ Composition

- Room class ไม่ได้มีกลไกป้องกันไม่ให้สร้าง object อิสระ
- Composition ไม่ใช่กฎที่ Python บังคับในระดับภาษา แต่เป็นวินัยในการออกแบบ interface ที่ผู้เขียน code ต้องรักษาไว้เอง
  - โดยการไม่เปิด constructor หรือ method ใดที่รับ object สำเร็จรูปของส่วนประกอบเข้ามาจากภายนอก

```python
class House:
    def __init__(self, address: str):
        self.address = address
        self.__rooms: list[Room] = []

    # เวอร์ชัน Composition (ที่ถูกต้องตามเจตนาของ House/Room)
    def add_room(self, name: str, area: float) -> None:
        room = Room(name, area)       # สร้างเอง — Room ผูกกับ House นี้ตลอดไป
        self.__rooms.append(room)

    # เวอร์ชัน Aggregation (ถ้าออกแบบผิด จะเปิดช่องให้ Room ลอยได้)
    def add_existing_room(self, room: Room) -> None:
        self.__rooms.append(room)     # รับจากภายนอก — Room อาจถูกใช้ซ้ำที่อื่นได้
```

- ถ้า House เปิด method แบบที่สอง (add_existing_room) ไว้ด้วย ความสัมพันธ์นี้จะกลายเป็น Aggregation ทันที
  - เปิดโอกาสให้ Room ถูกสร้างจากภายนอกแล้วนำมาผูกทีหลังได้
- ในกรณีของ House/Room ที่ต้องการให้เป็น Composition แท้จริง ควรมีแค่ add_room(name, area) เพียงแบบเดียว

สรุปเปรียบเทียบทั้ง 3 ความสัมพันธ์

	Association	Aggregation	Composition
เก็บเป็น attribute ถาวรหรือไม่	ไม่	ใช่	ใช่
Object ถูกสร้างจากไหน	ภายนอก (ใช้ชั่วคราว)	ภายนอก (ส่งเข้ามา)	ภายใน (สร้างเอง)
Lifetime เมื่อเจ้าของหายไป	ไม่เกี่ยวข้องกันอยู่แล้ว	ยังมีชีวิตอยู่ต่อได้	หมดความหมายไปด้วย
ตัวอย่าง	`BankAccount.transfer()`	`Team` กับ `Player`	`House` กับ `Room`

common mistake

ลืม Encapsulation เมื่อเก็บ List ของ Object

```python
class Team:
    def __init__(self, name: str):
        self.name = name
        self.players: list[Player] = []   # public — ลืม Encapsulation

    def add_player(self, player: Player) -> None:
        self.players.append(player)
```

ปัญหาจะปรากฏทันทีเมื่อโค้ดภายนอกเข้าถึง players โดยตรง

```python
team = Team("Barcelona")
team.add_player(Player("Messi", "Forward"))

team.players.append(fake_player)       # เพิ่มผู้เล่นได้โดยไม่ผ่าน add_player() เลย
team.players.clear()                   # ลบผู้เล่นทั้งหมดได้ในบรรทัดเดียว
team.players[0] = "ไม่ใช่ Player เลย"   # ยัดค่าอะไรก็ได้เข้าไปแทนที่
```

โค้ดภายนอกสามารถยัด string หรือค่าอะไรก็ได้เข้าไปแทนที่ Player object จริง
ทำให้ระบบทั้งหมดพังทันทีที่มีใครพยายามเรียก method ของ Player กับสิ่งที่ไม่ใช่ Player

ลืม Encapsulation เมื่อเก็บ List ของ Object (2)

วิธีแก้คือทำให้ _players เป็น private เสมอ และถ้าจำเป็นต้องเปิดให้อ่านค่าได้ ควรคืน copy ของ list แทนที่จะคืน reference ตรงๆ

```python
class Team:
    def __init__(self, name: str):
        self.name = name
        self.__players: list[Player] = []   # private

    def add_player(self, player: Player) -> None:
        self.__players.append(player)

    @property
    def players(self) -> list[Player]:
        return list(self.__players)   # คืน copy — แก้ไข list ที่ได้ไปไม่กระทบต้นฉบับ
```

ทำให้ Object ตัวเดิมกลายเป็น Object ใหม่โดยไม่ตั้งใจ

- code ที่เขียนผิดยังคงรันได้โดยไม่มี error ใดๆ เลย แต่ผลลัพธ์กลับผิดจากที่ตั้งใจไว้อย่างเงียบๆ
- ลองดู method ที่ ดูเหมือนถูกต้อง แต่จริงๆ แล้วทำลายเจตนาของ Aggregation

```python
class Team:
    def transfer_player_to(self, player: Player, new_team: "Team"):
        # ผิด — สร้าง Player ใหม่ขึ้นมาแทนที่จะย้าย object เดิม
        new_player = Player(player.name, player.position)
        new_team.add_player(new_player)
        self.remove_player(player)
```

- new_player = Player(player.name, player.position)
  - สร้าง Player object ใหม่ทั้งหมด โดยคัดลอกมาเฉพาะ name และ position แต่ไม่ได้คัดลอกสถานะภายในอย่าง goals ที่สะสมไว้
- ผลลัพธ์คือ “Messi คนใหม่” ที่มีชื่อและตำแหน่งเหมือนเดิม
  - แต่ประวัติการทำประตูหายไปหมดราวกับเป็นคนละคน

```python
messi = Player("Messi", "Forward")
barcelona = Team("Barcelona")
barcelona.add_player(messi)

messi.score_goal()
messi.score_goal()
print(messi.goals)    # 2

inter_miami = Team("Inter Miami")
barcelona.transfer_player_to(messi, inter_miami)

# ตรวจสอบผู้เล่นในทีมใหม่
new_messi = inter_miami.players[0]
print(new_messi.goals)    # 0 — ข้อมูลหายไปหมด!
```

ทำให้ Object ตัวเดิมกลายเป็น Object ใหม่โดยไม่ตั้งใจ (2)

- วิธีแก้คือ ส่ง object ตัวเดิมไปตรงๆ ไม่สร้างใหม่

```
def transfer_player_to(self, player: Player, new_team: "Team")
    new_team.add_player(player)    # ส่ง object เดิม ไม่สร้างใหม่
    self.remove_player(player)
```

```
barcelona.transfer_player_to(messi, inter_miami)
new_messi = inter_miami.players[0]
print(new_messi.goals)          # 2 — ข้อมูลยังอยู่ครบ
print(new_messi is messi)       # True — เป็น object เดียวกันเป๊ะ
```

- เมื่อมี object อยู่ในมือ ไม่ควรสร้างใหม่โดยไม่จำเป็น โดยเฉพาะในบริบทของ Aggregation ที่เจตนาให้ object เดิมถูกใช้ซ้ำและย้ายไปมาได้
- การสร้างใหม่ทุกครั้งขัดกับเจตนานี้โดยตรงและทำให้สูญเสียสถานะที่สะสมไว้ โดยไม่มี error ใดเตือน

สับสนระหว่าง “has-a” กับ “is-a”

- ระบบ Playlist/Song ถ้านิสิตพยายามลัดขั้นตอนโดยให้ Playlist inherit จาก list แทนที่จะเก็บ list ไว้เป็น attribute

```
class Playlist(list):
    """ผิด — Playlist ไม่ใช่ list แต่ "มี" list ของเพลง"""
    def __init__(self, name: str):
        super().__init__()
        self.name = name
```

```
playlist = Playlist("เพลงโปรด")
playlist.append(song1)      # ใช้ method ของ list ได้เลย ดูเหมือนสะดวก
playlist.append(song2)

playlist.sort()             # ← อันตราย! ไม่ควรมี method นี้ใน Playlist เลย
playlist.reverse()          # ← เช่นกัน
playlist.pop()              # ← เช่นกัน ลบเพลงออกโดยไม่ผ่าน logic ใดๆ ของ Playlist
```

- Playlist ไม่ผ่าน is-a test เพราะ Playlist ไม่ใช่ list ทั่วไป แต่ “มี” list ของเพลงอยู่ภายใน
- การ inherit จาก list ทำให้ Playlist ได้รับ method ที่ไม่เกี่ยวข้องหรือไม่ควรมีมาโดยอัตโนมัติ เช่น sort(), reverse(), pop(), insert()
- เปิดช่องให้แก้ไข playlist ในรูปแบบที่ผู้ออกแบบระบบไม่ได้ตั้งใจให้ทำได้

สับสนระหว่าง “has-a” กับ “is-a” (2)

- ต้องใช้ Aggregation คือ ให้ Playlist เก็บ list ของ Song ไว้เป็น private attribute

```python
class Playlist:
    """ถูกต้อง — Playlist มี list ของเพลง ไม่ใช่ list เอง"""
    def __init__(self, name: str):
        self.name = name
        self.__songs: list[Song] = []

    def add_song(self, song: Song) -> None:
        self.__songs.append(song)

    def remove_song(self, song: Song) -> None:
        if song in self.__songs:
            self.__songs.remove(song)

    @property
    def song_count(self) -> int:
        return len(self.__songs)
```

- ผู้ใช้งาน Playlist ทำได้เฉพาะสิ่งที่ผู้ออกแบบตั้งใจ (add_song(), remove_song()) เท่านั้น
- ไม่มีทางเรียก playlist.sort() หรือ playlist.pop() ได้ เพราะ method เหล่านั้นไม่เคยมีอยู่ใน Playlist เลยตั้งแต่ต้น

แบบฝึกหัด

สถานการณ์ที่ 1 – Playlist กับ Song

- ระบบเพลงมี Playlist ที่เก็บรายการเพลง โดยมี
  - method add_song(song: Song) ที่รับ Song object เข้ามา
  - เพลงเดียวกันสามารถถูกเพิ่มเข้าไปในหลาย playlist พร้อมกันได้ (เช่น เพลงหนึ่งอยู่ทั้งใน “เพลย์ลิสต์โปรด” และ “เพลย์ลิสต์ออกกำลังกาย”)
- ลองคิดดู: ความสัมพันธ์นี้ควรเป็นแบบใด

สถานการณ์ที่ 2 — Invoice กับ InvoiceLine

- ระบบใบแจ้งหนี้มี Invoice ที่มี method add_line(product_name: str, quantity: int, unit_price: float) รับเฉพาะข้อมูลดิบเข้ามา ไม่มีทางสร้าง InvoiceLine object แล้วส่งเข้าไปได้เลย
- ลองคิดดู: ความสัมพันธ์นี้ควรเป็นแบบใด

สถานการณ์ที่ 3 – Doctor กับ Patient

- ระบบโรงพยาบาลมี Doctor ที่มี method examine(patient: Patient) ใช้สำหรับตรวจคนไข้
- โดยภายใน method นี้ไม่มีการเก็บ patient ไว้เป็น attribute ของ Doctor เลย เพียงแค่บันทึกผลการตรวจลงใน patient object แล้วจบการทำงาน
- ลองคิดดู: ความสัมพันธ์นี้ควรเป็นแบบใด

สถานการณ์ที่ 4 – Car กับ Engine

- ระบบยานพาหนะมี Car ที่สร้าง Engine ขึ้นภายใน _init_ ของตัวเองเสมอ (self.engine = Engine(horsepower))
  - ไม่มีทางเปลี่ยน Engine ของรถคันหนึ่งไปใช้กับรถอีกคันได้เลยในระบบนี้
- ลองคิดดู: ความสัมพันธ์นี้ควรเป็นแบบใด

สถานการณ์ที่ 5 – University กับ Department

- ระบบมหาวิทยาลัยมี University ที่เก็บ Department (คณะ) หลายคณะ โดยแต่ละ Department ถูกสร้างขึ้นตอนก่อตั้งมหาวิทยาลัยเท่านั้น
  - ไม่มีการย้าย Department ไปมหาวิทยาลัยอื่น และถ้ามหาวิทยาลัยปิดตัวลง คณะทั้งหมดก็ถือว่ายุบไปด้วย
- ลองคิดดู: ความสัมพันธ์นี้ควรเป็นแบบใด